

T1 – Servidor HTTP/1.1 sobre sockets TCP

Livia Noer
Luthero Vargas
Mariana Adam
Nicolas Borges

Angelo Elias Zotto
PUCRS - Escola Politécnica

Servidor HTTP/1.1 em Python 3 (só biblioteca padrão), escrito diretamente sobre a API de sockets TCP. As medições foram feitas entre máquinas distintas ligadas por Wi-Fi: nas capturas C1, C2, de segurança e da transação completa, o cliente é 192.168.18.221 e o servidor 192.168.18.220; no teste de atendimento simultâneo, o servidor é .221 e os clientes são .220 e .219. As capturas estão em `medicoes/capturas/`.

Descrição da arquitetura

O servidor (`src/server.py`) tem três partes:

- **Aceitação** (`serve`): socket TCP com bind em `0.0.0.0:<porta>` e um loop de `accept()`; cada conexão aceita vai para uma thread nova.
- **Leitura e parsing** (`read_head`, `parse_request`, `resolve_path`): como o TCP entrega um fluxo de bytes, os dados são acumulados em um buffer até o `\r\n\r\n`; o que sobra fica guardado para a próxima requisição da mesma conexão. O caminho é decodificado (*percent-decoding*) antes da checagem de travessia.
- **Resposta** (`send_response`): monta a linha de status e os headers `Date` (IMF-fixdate), `Server`, `Content-Type`, `Content-Length` e `Connection`, e envia tudo com `sendall()`. No HEAD, os headers são os mesmos do GET, sem o corpo.

Concorrência: uma thread por conexão. O servidor passa quase todo o tempo esperando rede (`recv/send`), e essas chamadas liberam o GIL do Python, então as threads avançam em paralelo. Um cliente lento bloqueia só a sua própria thread. A solução é simples, não precisa de locks (as threads não compartilham estado mutável) e atende bem a quantidade de conexões deste trabalho.

Conexões persistentes (Parte 2). Em HTTP/1.1 a conexão fica aberta por padrão; se o cliente envia `Connection: close`, o servidor responde com o mesmo header e fecha. Cada conexão atende várias requisições em sequência no mesmo loop, e `conn.settimeout(5)` fecha a conexão após 5 s sem atividade.

Tabela de conformidade

Saída completa em `medicoes/resultados/evidencias/curl_conformidade_e_seguranca.txt`.

Requisição	Resposta obtida
curl -i http://<ip>:8080/index.html	200 OK, Content-Length: 414
curl -I http://<ip>:8080/index.html	200 OK, Content-Length: 414, sem corpo
curl -i http://<ip>:8080/naoexiste.html	404 Not Found
curl -i -X DELETE http://<ip>:8080/index.html	405 Method Not Allowed, Allow: GET, HEAD
Request line sem versão: printf 'GET /index.html\r\nHost: <ip>\r\nConnection: close\r\n\r\n' nc <ip> 8080	400 Bad Request
Travessia de diretório	403 Forbidden (próxima seção)

Exemplo de resposta completa (GET /index.html, corpo omitido):

```
HTTP/1.1 200 OK
Date: Sun, 04 Oct 2026 21:33:59 GMT
Server: T1-Lab-Redes/1.0
Content-Type: text/html; charset=utf-8
Content-Length: 414
Connection: keep-alive
```

Demonstração de segurança

O servidor rejeita qualquer segmento .. depois de decodificar o caminho e, como segunda barreira, confere com `os.path.realpath` que o arquivo final está dentro do diretório raiz. As três tentativas abaixo foram feitas com `curl --path-as-is` de outra máquina (`testes403.pcapng`) e tentam ler `src/server.py`, que fica fora de `www/`:

Requisição	Resposta
GET ../../src/server.py	403 Forbidden
GET /%2E%2E/src/server.py	403 Forbidden
GET /%2E%2E%2Fsrc%2Fserver.py	403 Forbidden

Resposta capturada (idêntica nas três):

```
HTTP/1.1 403 Forbidden
Date: Sun, 04 Oct 2026 05:53:23 GMT
Server: T1-Lab-Redes/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 14
Connection: keep-alive

403 Forbidden
```

Captura de uma transação completa

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.18.221	192.168.18.220	TCP	66	63498 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
2	0.210523800	192.168.18.220	192.168.18.221	TCP	66	8080 → 63498 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 SACK_PERM WS=512
3	0.210667700	192.168.18.221	192.168.18.220	TCP	54	63498 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0
4	0.210930400	192.168.18.221	192.168.18.220	HTTP	123	GET /index.html HTTP/1.1
5	0.215557600	192.168.18.220	192.168.18.221	TCP	60	8080 → 63498 [ACK] Seq=1 Ack=70 Win=65536 Len=0
6	0.219823600	192.168.18.220	192.168.18.221	HTTP	630	HTTP/1.1 200 OK (text/html)
7	0.219824600	192.168.18.220	192.168.18.221	TCP	60	8080 → 63498 [FIN, ACK] Seq=577 Ack=70 Win=65536 Len=0
8	0.219918700	192.168.18.221	192.168.18.220	TCP	54	63498 → 8080 [ACK] Seq=70 Ack=578 Win=64768 Len=0
9	0.220123500	192.168.18.221	192.168.18.220	TCP	54	63498 → 8080 [FIN, ACK] Seq=70 Ack=578 Win=64768 Len=0
11	0.223061900	192.168.18.220	192.168.18.221	TCP	60	8080 → 63498 [ACK] Seq=578 Ack=71 Win=65536 Len=0

Primeira conexão de C1.pcapng, cliente .221 e servidor .220:

- **Handshake:** frames 1–3 (SYN, SYN, ACK, ACK).
- **Requisição:** frame 4, GET /index.html com Connection: close.
- **Resposta:** frame 5 (ACK do servidor) e frame 6 (HTTP/1.1 200 OK com os 414 bytes do HTML).
- **Encerramento:** frame 7 (FIN do servidor), 8 (ACK do cliente) e 9 (FIN do cliente). O último ACK do servidor aparece logo depois, já que o frame 10 é o SYN da próxima conexão.

Evidência de atendimento simultâneo

Em teste2clientesWiresharkServer.pcapng, dois clientes (.220 e .219) carregam a página de teste ao mesmo tempo. Trecho das requisições e respostas (tempo em segundos):

```
0,0057 cliente .220 -> servidor GET /
0,0279 cliente .219 -> servidor GET /
0,0365 cliente .220 -> servidor GET /style.css
0,0365 cliente .219 -> servidor GET /style.css
0,0379 servidor -> cliente .220 200 /style.css
0,0380 servidor -> cliente .219 200 /style.css
```

As requisições dos dois chegam intercaladas, com os GET /style.css a 3 μ s de distância, e ambos recebem 200 OK para todos os recursos. As seis conexões abertas pelos dois navegadores ficam ativas ao mesmo tempo e são fechadas pelo servidor após os 5 s de ociosidade.

RTT medido

Fonte	RTT
ping -n 10 (.221 → .220)	média 71 ms (mín. 4, máx. 115)
Handshakes das conexões 2–10 do C1 (SYN → SYN-ACK)	média 5,1 ms (3,0 a 11,2)
Primeiro handshake de cada captura	81 a 210 ms

O Wi-Fi variou muito durante o ping, e a primeira conexão de cada sessão foi sempre lenta. Por isso a análise usa o RTT de 5,1 ms, medido nos próprios handshakes do C1 durante a medição.

Tabela comparativa C1 vs C2

10 requisições sequenciais a /index.html; C1 abre uma conexão por requisição (Connection: close), C2 usa uma conexão persistente. Bytes = tamanho dos frames na captura.

Métrica	C1	C2	Economia
Handshakes TCP	10	1	90 %
Pacotes	100	28	72,0 %
Bytes	12 270	8 094	34,0 %
Tempo total	377,0 ms	200,2 ms	46,9 %

Análise do overhead de conexão

Cada conexão do C1 gasta 3 pacotes (186 B) no handshake e 4 pacotes (228 B) no encerramento: 7 pacotes e 414 B sem nenhum dado HTTP. Nas 10 conexões, são **70 pacotes (70 % do total) e 4 140 B (33,7 %)** só para abrir e fechar conexões. No C2 esse custo é pago uma vez só.

Análise em função do RTT

Cada conexão nova custa 1 RTT de handshake antes de a requisição poder ser enviada, e cada requisição custa mais 1 RTT. Assim, o C1 gasta $10 \times 2 = 20$ RTT e o C2 gasta $1 + 10 = 11$ RTT: **9 RTTs a mais**, um por handshake das 9 conexões extras.

Na captura, esses 9 handshakes somam 46,1 ms ($9 \times 5,1$ ms). A diferença total entre os cenários foi maior (176,8 ms) porque o primeiro handshake do C1 demorou 210 ms, contra 101 ms no C2; essa diferença de 109 ms é efeito do Wi-Fi na primeira conexão, não da persistência. O encerramento não acrescenta tempo, pois o cliente já abre a conexão seguinte logo após enviar seu FIN.

Conclusão

A conexão persistente evita um handshake (1 RTT) por requisição. Aqui isso economizou 72 % dos pacotes e 46 ms nos handshakes, porque o RTT da rede local é de cerca de 5 ms. O ganho cresce com o RTT e com o número de conexões evitadas: com um RTT de 100 ms, típico de um servidor distante na internet, os mesmos 9 handshakes custariam cerca de 0,9 s, enquanto o C2 continuaria pagando um único handshake.